

CS342 Machine Learning: Latent Factor Models with Kernels

Archie Harrodine

December 2023

1 Task 1

We want to perform Principal Component Analysis (PCA) on a dataset which has had a change of basis, without explicitly knowing or calculating the mapping. We know that:

$$\mathbf{Z} = \phi(\mathbf{X}) \mid \mathbf{X} \in \mathbb{R}^{n \times d}, \mathbf{Z} \in \mathbb{R}^{n \times m} \quad (1)$$

Normally to perform PCA on $\mathbf{Z}$, we would need to use Singular Value Decomposition to find its top PCs:

$$\mathbf{Z} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \mid \mathbf{U} \in \mathbb{R}^{n \times n}, \mathbf{\Sigma} \in \mathbb{R}^{n \times m}, \mathbf{V} \in \mathbb{R}^{n \times m} \quad (2)$$

We want to isolate $\mathbf{Z}$'s top k components, found with $\mathbf{V}^T = \mathbf{W}$, and then project our dataset using $\mathbf{Z}' = \mathbf{Z}\mathbf{W}$, where $\mathbf{Z}'$ is our projected data. We can do this using our kernel. We know that:

$$\mathbf{K} = \phi(\mathbf{X})\phi(\mathbf{X})^T = \mathbf{Z}\mathbf{Z}^T \mid \mathbf{K} \in \mathbb{R}^{n \times n} \quad (3)$$

where $\mathbf{K}$ is our Gram matrix such that $\mathbf{K}_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j) \forall i, j \in [1, n]$

We are going to assume that we can center both $\mathbf{Z}$ and $\mathbf{K}$.

Now with this set up we can find the relation between $\mathbf{Z}$ and $\mathbf{K}$.

$$\begin{aligned} \mathbf{K} &= \phi(\mathbf{X})\phi(\mathbf{X})^T \\ &= \mathbf{Z}\mathbf{Z}^T \\ &= (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T)(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T)^T \quad \text{by (2)} \\ &= (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T)\mathbf{V}^{TT}(\mathbf{U}\mathbf{\Sigma})^T \\ &= \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T\mathbf{V}\mathbf{\Sigma}^T\mathbf{U}^T \end{aligned}$$

We know that $\mathbf{V}$ is orthonormal, hence $\mathbf{V}^T = \mathbf{V}^{-1}$

$$\begin{aligned} &= \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{-1}\mathbf{V}\mathbf{\Sigma}^T\mathbf{U}^T \\ &= \mathbf{U}\mathbf{\Sigma}\mathbf{I}\mathbf{\Sigma}^T\mathbf{U}^T \\ &= \mathbf{U}\mathbf{\Sigma}\mathbf{\Sigma}^T\mathbf{U}^T \end{aligned}$$

We know that $\Sigma \in \mathbb{R}^{n \times m}$ and is diagonal matrix with padding.

$$\Sigma = \underbrace{\begin{bmatrix} \sigma_1 & 0 & \dots & 0 \\ 0 & \sigma_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \sigma_m \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix}}_m \bigg\}^n$$

$$\Sigma \Sigma^T = \Sigma^2 = \underbrace{\begin{bmatrix} \sigma_1^2 & & & & \\ & \sigma_2^2 & & & \\ & & \ddots & & \\ & & & \sigma_m^2 & \\ & & & & \ddots \\ & & & & & 0 \end{bmatrix}}_n \bigg\}^n \quad (4)$$

This assumes $n > m$, however if this is not the case, it just means we have n values and padding in the columns instead of the rows, with negligible impact.

We also know that $\mathbf{U}$ is orthonormal, hence $\mathbf{U}^T = \mathbf{U}^{-1}$

This gives us that:

$$\mathbf{K} = \mathbf{U} \Sigma^2 \mathbf{U}^{-1} \quad (5)$$

If we now look at $\mathbf{K}$ separately, we can choose to diagonalise it, which gives:

$$\mathbf{K} = \mathbf{D} \mathbf{\Lambda} \mathbf{D}^{-1} \quad (6)$$

Such that $\mathbf{D}$ are the eigenvectors of $\mathbf{K}$ and $\mathbf{\Lambda}$ is a diagonal matrix containing the eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$ of $\mathbf{K}$

$$\begin{aligned} \mathbf{U} \Sigma^2 \mathbf{U}^{-1} &= \mathbf{D} \mathbf{\Lambda} \mathbf{D}^{-1} \\ \implies \mathbf{U} &= \mathbf{D} \end{aligned} \quad (7)$$

$$\begin{aligned} \Sigma^2 &= \mathbf{\Lambda} \\ \implies \sigma_i &= \sqrt{\lambda_i} \quad \forall i = 1 \dots m \end{aligned} \quad (8)$$

We can conclude this since the dimensions of $\mathbf{D}$ and $\mathbf{U}$ are equal, and the dimensions of $\mathbf{\Lambda}$ and Σ^2 are equal. This means we can piecewise associate the values, and conclude (7) and (8).

By the lectures we know that our projection $\mathbf{Z}' = \mathbf{Z}\mathbf{W} = \mathbf{U}\mathbf{\Sigma}$. We have found that $\mathbf{U} = \mathbf{D}$ and that $\mathbf{\Sigma} = \sqrt{\mathbf{\Lambda}}$ such that $\sqrt{\mathbf{\Lambda}}$ is an $n \times m$ diagonal matrix with values $\sqrt{\lambda_1}, \sqrt{\lambda_2}, \dots, \sqrt{\lambda_m}$. Both $\mathbf{D}$ and $\sqrt{\lambda_1}, \sqrt{\lambda_2}, \dots, \sqrt{\lambda_m}$ are calculated from $\mathbf{K}$, thus we have performed PCA on $\mathbf{Z} = \phi(\mathbf{X})$ without calculating $\mathbf{Z}$.

Next we need to project the top k PCs without explicitly calculating them; as in without calculating $\mathbf{V}^T = \mathbf{W}$. From the lectures, we know we can find the top k PCs using only $\mathbf{U}$ and $\mathbf{\Sigma}$. We order the pairs of columns of $\mathbf{U}$ and the values of $\mathbf{\Sigma}$ in descending order: $\sigma_1 > \sigma_2 > \dots > \sigma_m$. We then take the top k and calculate columns of $\mathbf{W}$ (our PCs) using:

$$\mathbf{w}_c = \frac{1}{\sigma_c} \mathbf{Z}^T \mathbf{u}_c \quad (9)$$

We want to calculate the projection $\mathbf{Z}' = \mathbf{Z}\mathbf{W}$, so we will premultiply (9) with $\mathbf{Z}$.

$$\begin{aligned} \mathbf{Z}\mathbf{w}_c &= \frac{1}{\sigma_c} \mathbf{Z}\mathbf{Z}^T \mathbf{u}_c \\ &= \frac{1}{\sigma_c} \mathbf{K}\mathbf{u}_c \quad \text{by (4)} \end{aligned} \quad (10)$$

This gives a $n \times 1$ projection of the c th component, which if we combine with the top k components we get:

$$\mathbf{Z}' = \left[\underbrace{\begin{bmatrix} | & | & & | \\ \mathbf{Z}\mathbf{w}_1 & \mathbf{Z}\mathbf{w}_2 & \dots & \mathbf{Z}\mathbf{w}_k \\ | & | & & | \end{bmatrix}}_k \right] \Bigg\}^n \quad (11)$$

Since we have $\mathbf{U}$ and $\mathbf{\Sigma}$ calculated through $\mathbf{K}$, and we calculate the projection without ever having to calculate $\mathbf{W}$ (the PCs), using (10) we have proven we can project the data points in $\mathbf{Z} = \phi(\mathbf{X})$ without explicitly using the PCs.

2 Task 2

The homogeneous polynomial kernel is used to project data to a higher dimensionality, which introduces non-linearity into the feature space. If our data is linearly inseparable, we may be able to separate it using a non linear separator, such as a quadratic function. By projecting our data to a higher dimensionality, we work with the data fit to a new feature space; meaning that non linear functions can be represented linearly in this dimension. Applying this to our task, we would try to use the homogeneous polynomial kernel to wrap a circular separator around our data (Fig:1).

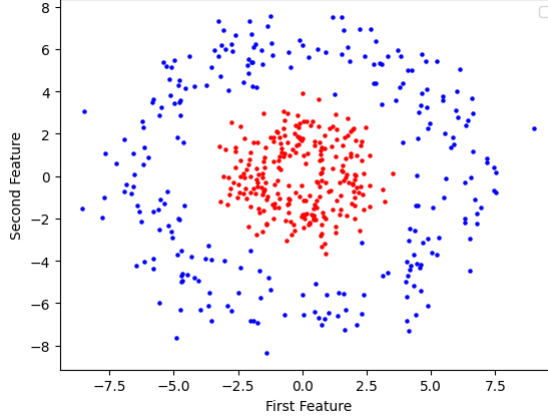


Figure 1: Original features with their classifications shown by colors.

As we are working with only a homogeneous kernel, we are limited to how successful working in higher dimensions will be, as the new dimensions added will only be able to work within a certain shape. We would be altering the hyper parameter of d which refers to the new dimension we are raising the data to. Additionally, it is important to note that as there is no assertion that $\mathbf{K}$ is positive, we can only use integer values for the hyper parameter, otherwise complex values are given which are not useful for classification. We experiment with the hyper parameter d of the homogeneous polynomial using our PCA implementation, and look at the top 3 principal components (as they can be visualised easily) to see the potential of any linearly separable data projections. Given (Fig:6) we can see that $d = 1$ does nothing to meaningfully change the data, as it projecting the data into the same number of dimensions, so the circular shape is maintained. We can see that odd valued d gives us bundles of class 1 surrounded by class 2, acting similarly to the original dataset with greater variation in the datapoints surrounding the center - which follows from $d = 1$ also being odd. Furthermore for odd d , $\text{sgn}(x) = \text{sgn}(x^d)$ meaning the projection stretches and shrinks distances between points, but never causes them to wrap around to a new direction.

When looking at even d , the results look far more promising as we see classes grouping together to spread out as a cone, which follows from the approximate shape of even degree polynomials. As this is homogeneous, we form the cone at the center of the original data, grouping much of the central class together. When we increase our d to larger values, we see that more and more data points collapse in on themselves; which is a result of $|\langle \mathbf{x}_i, \mathbf{x}_j \rangle| < 1$ so large d makes them increasingly smaller, thus increasingly similar. This shows that if we were to use homogeneous that we would want to use a small even value. At first $d = 2$ looks promising from the plots, however we see when looking at the 2D planes of each axis (Fig:2), that the plot has no clear separation. This is from a result of our ‘cone’ that we are forming is limited without the constant present in nonhomogeneous polynomials, and so we are forced to capture points which are close to the border. This makes homogeneous polynomial kernel not very suitable for this task.

The Gaussian radial basis function kernel is used to project data to into vector space of infinite dimension. The function acts as a density function of Euclidean distance similarities (Gaussian

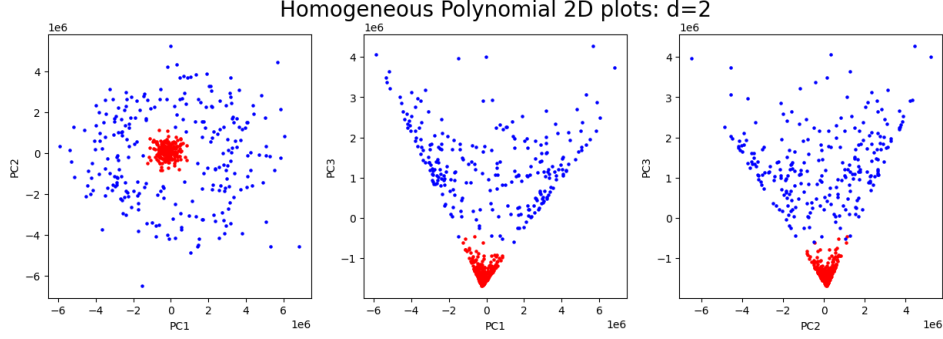


Figure 2: 2D plots of homogeneous polynomial kernel where $d = 2$.

function). This can be interpreted from the function:

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{1}{2\sigma^2} \|\mathbf{x}_i - \mathbf{x}_j\|^2\right) \quad (12)$$

This function uses the Gaussian function using the Euclidean distance to weigh how close each point is. Closer vectors are rewarded higher values as their Euclidean distance will be closer to 0. The influence of each datapoint is determined by the hyper parameter σ , which influences the range of what is deemed ‘close’ by the kernel. With larger values of σ , the conceptual Gaussian bell curve gets wider, which extends the impact of a single datapoint, effectively extending the reach of what vectors are considered close, and which aren’t.

Studying how the plot of top 3 PCs changes with this hyper parameter showcases this shift (Fig:7)¹. For small values of σ , class 2 data has collapsed in on itself. This is because there exist large distances between datapoints in class 2 (Fig:1). Then when looking at the radial basis function, if distance δ is large and σ is small, $\exp -\frac{\delta^2}{2\sigma^2}$ will be very small, as σ is too small to stop δ from dominating. This remains between $\sigma = 0.01$ and $\sigma = 1$. When we reach $\sigma = 2$ we find that σ is large enough that class 2 does not collapse in on itself, and instead a dome is forming between class 1 and class 2 datapoints. This continues as σ increases, as class 2 expands, and class 1 closes in on itself. Now σ is large, points with large Euclidean distances between them are properly represented, whereas close points are grouped together like a bell curve due to their similarity. This behavior is crucial for our classification as we want to be able to group together the data that is close together in the circle. As we approach $\sigma = 10$, we see a bell curve that has grouped together class 1 to the end; which implies we can linearly separate the data, which we can see by the straight line we can draw when looking at the 2D plots of the planes (Fig:3). If we keep increasing σ to larger numbers past 20, we see that class 1 data continues to bunch up closer and closer together, however remains to be linearly separable, showing that past a certain threshold, the data is effectively linearly separated.

We can actually derive approximately what this threshold is by looking at our original data. Since σ reflects the kernel’s ‘width’, this means that it is likely closely tied to the width of the original

¹More values between these hyper parameter values were used, but this selection best shows how the plot changes with σ

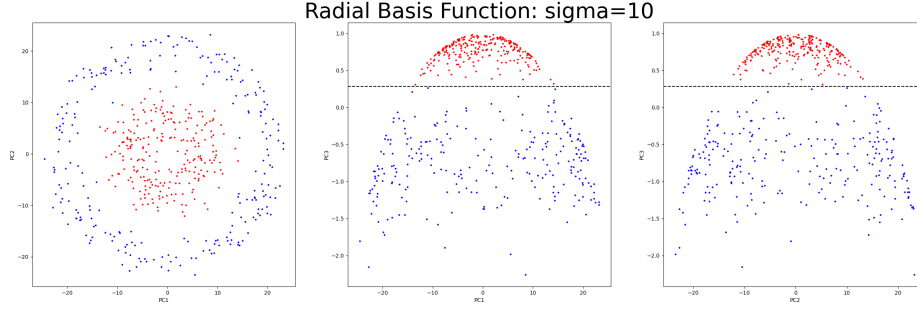


Figure 3: 2D plots of radial basis function kernel where $\sigma = 10$.

data we are trying to classify. Looking at class 1, and finding the average of the largest 10 Euclidean distances between points, we find that the approximate diameter (or width) of the center of class 1 datapoints is 7.169. When we look at how the plot changes between 6.8 and 7.2(Fig:6), we can see that not only does the data flip, but it also becomes much more clearly separable. The data seems to flip at approximately 7.05. This tells us that the width σ directly relates to the diameter of the cluster of data we want to classify. So long as our hyper parameter exceeds the threshold, we can linearly classify our data.

Hence to conclude, we will use Gaussian radial basis function kernel, with a provided $\sigma = 10$. We use the Gaussian kernel because of the shown inadequacy of the homogeneous polynomial, and because the Gaussian kernel implicitly projects groups of vectors which are similar, which suits this problem of classifying data that is in a cluster. We can choose $\sigma = 10$ as so long as it is greater than the approximate threshold 7.05, it will likely be viable. Specifically 10 is chosen as the experiments found this to be a successful hyper parameter to classify the data.

3 Task 3, 4 and 5

The minimum number of top PCs of the new data that we need is 3. This can be seen with (Fig:3) where on the second and third plots, we can see that we can linearly separate the two classes with a horizontal line of $PC3 = 0.286$. If we only had two principal components we would be limited to what is depicted in the first subplot, which does not help us. Since we can separate the two classes, this is our effective decision stump, as any piece of data with their 3rd PC > 0.286 is labelled class 1, and anything else is class 2. By implementing this code and running it in comparison with ground truth labels, we achieved 100% accuracy as seen in the code. We came to this separator by using our justifications of our kernel and hyper parameter choice, and then calculating the distance between the lowest of PC3 of class 1, and the highest of PC3 of class 2.

$$\begin{aligned} \text{midpoint} &= \frac{\max(\text{class1}) + \max(\text{class2})}{2} \\ &= \frac{0.31037890 + 0.26152499}{2} \\ &\approx 0.286 \end{aligned}$$

With the decision stump described, and its success of classification being 100%, we can now look into plotting the data. With our decision stump we identified that not only is the data linearly separable, but it is separable in just one dimension, with respect to $\mathbf{Z}$'s third PC. Because of this, we can plot the data in 1D(Fig:4), 2D(Fig:3), and 3D(Fig:5).

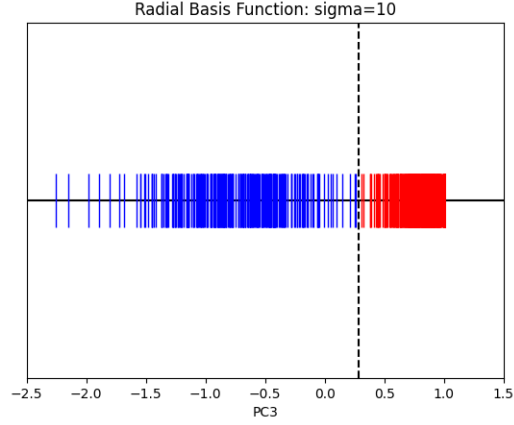


Figure 4: 1D plot of radial basis function kernel where $\sigma = 10$.

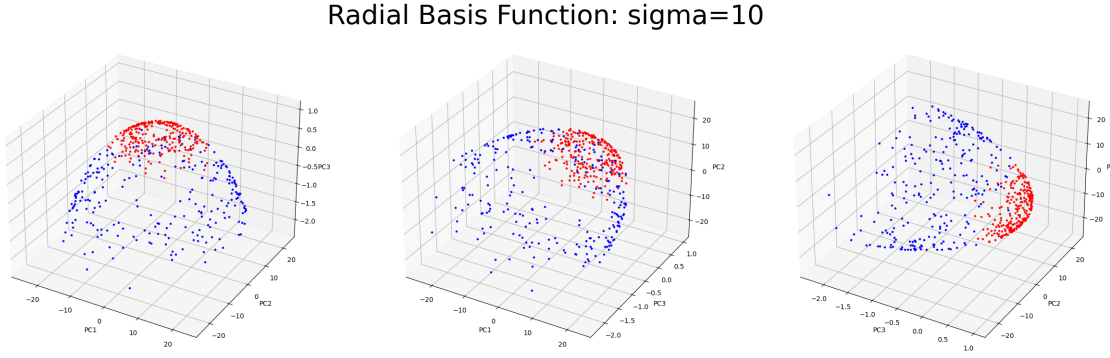
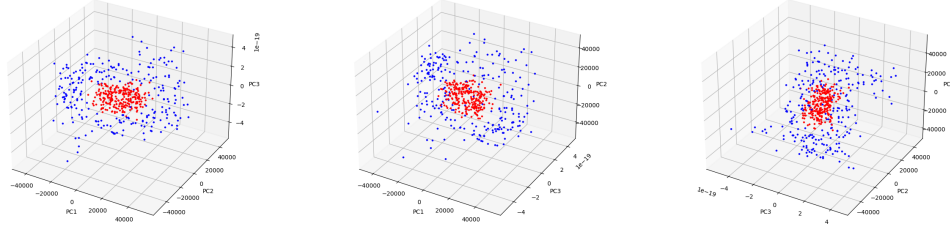
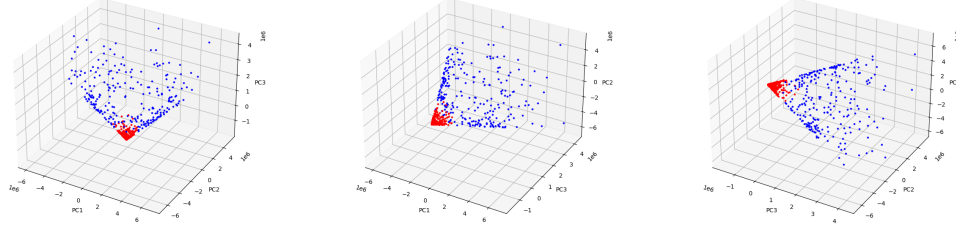


Figure 5: 3D plot of radial basis function kernel where $\sigma = 10$.

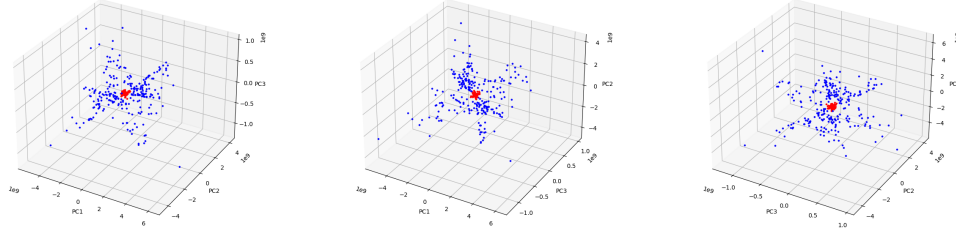
Homogeneous Polynomial: $d=1$



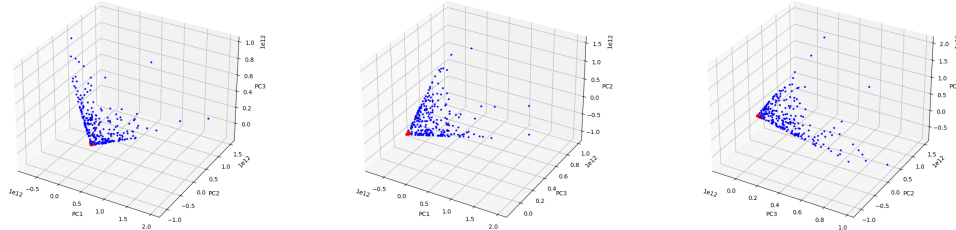
Homogeneous Polynomial: $d=2$



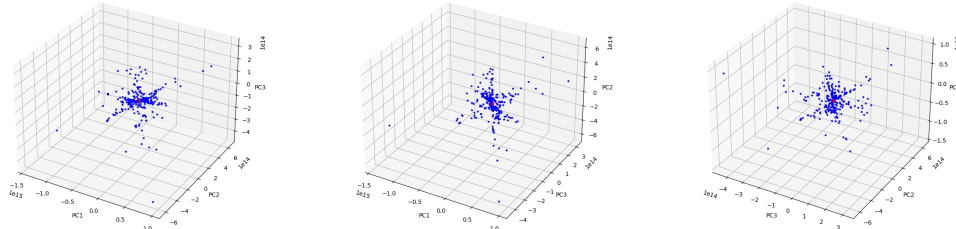
Homogeneous Polynomial: $d=3$



Homogeneous Polynomial: $d=4$



Homogeneous Polynomial: $d=5$



Homogeneous Polynomial: $d=6$

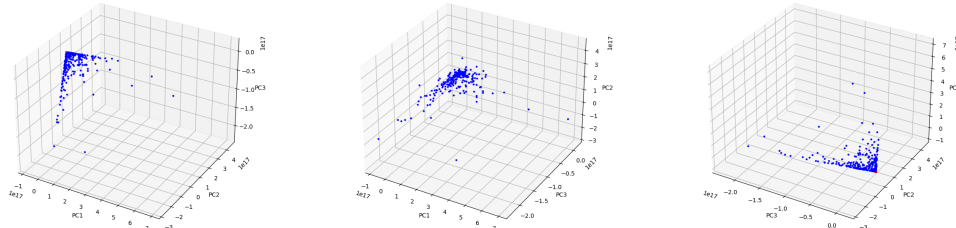
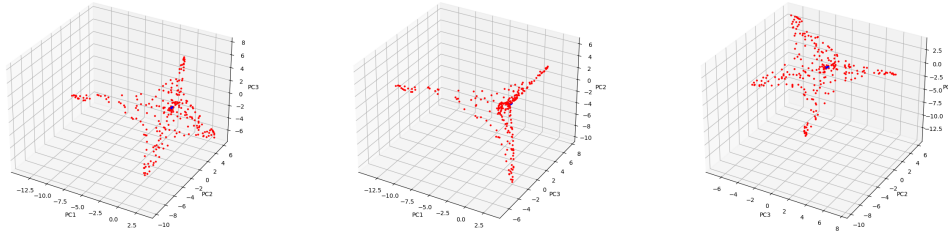
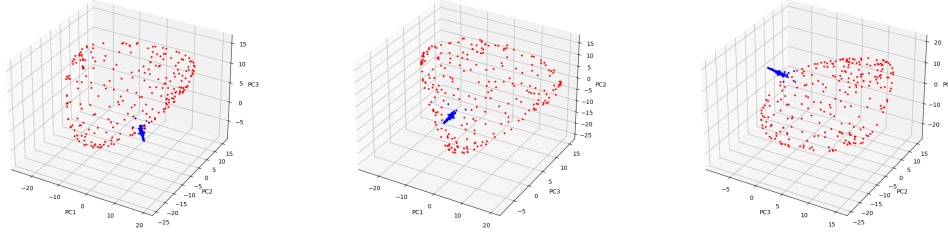


Figure 6: 3D plots of homogeneous polynomial kernels of different hyper parameters from multiple angles.

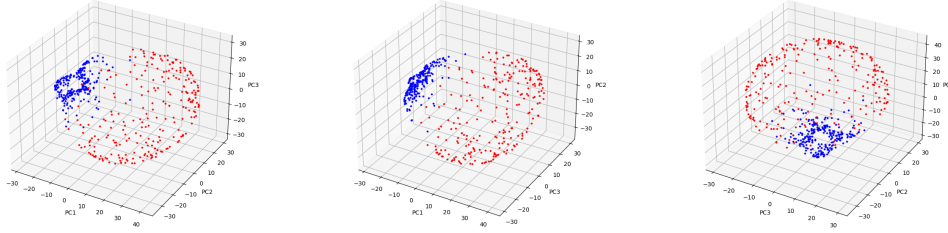
Radial Basis Function: sigma=0.5



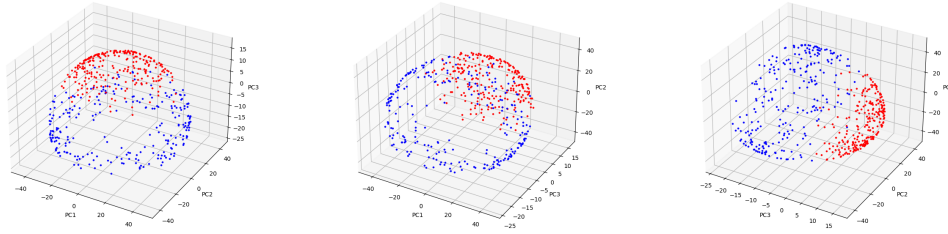
Radial Basis Function: sigma=1



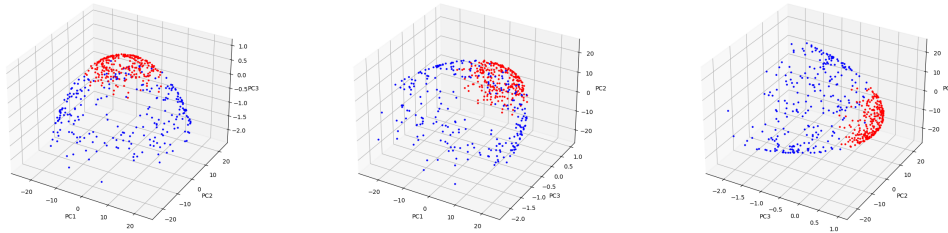
Radial Basis Function: sigma=2



Radial Basis Function: sigma=5



Radial Basis Function: sigma=10



Radial Basis Function: sigma=15

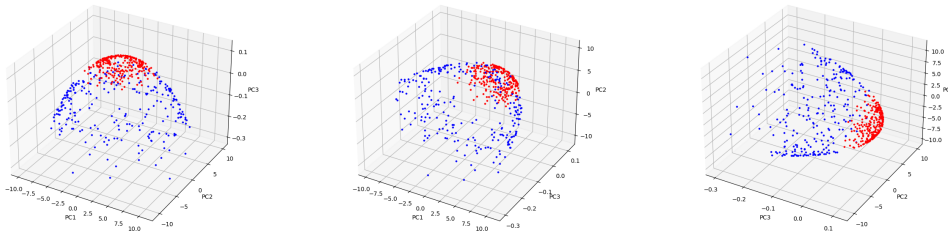


Figure 7: 3D plots of radial basis function kernels of different hyper parameters from multiple angles.

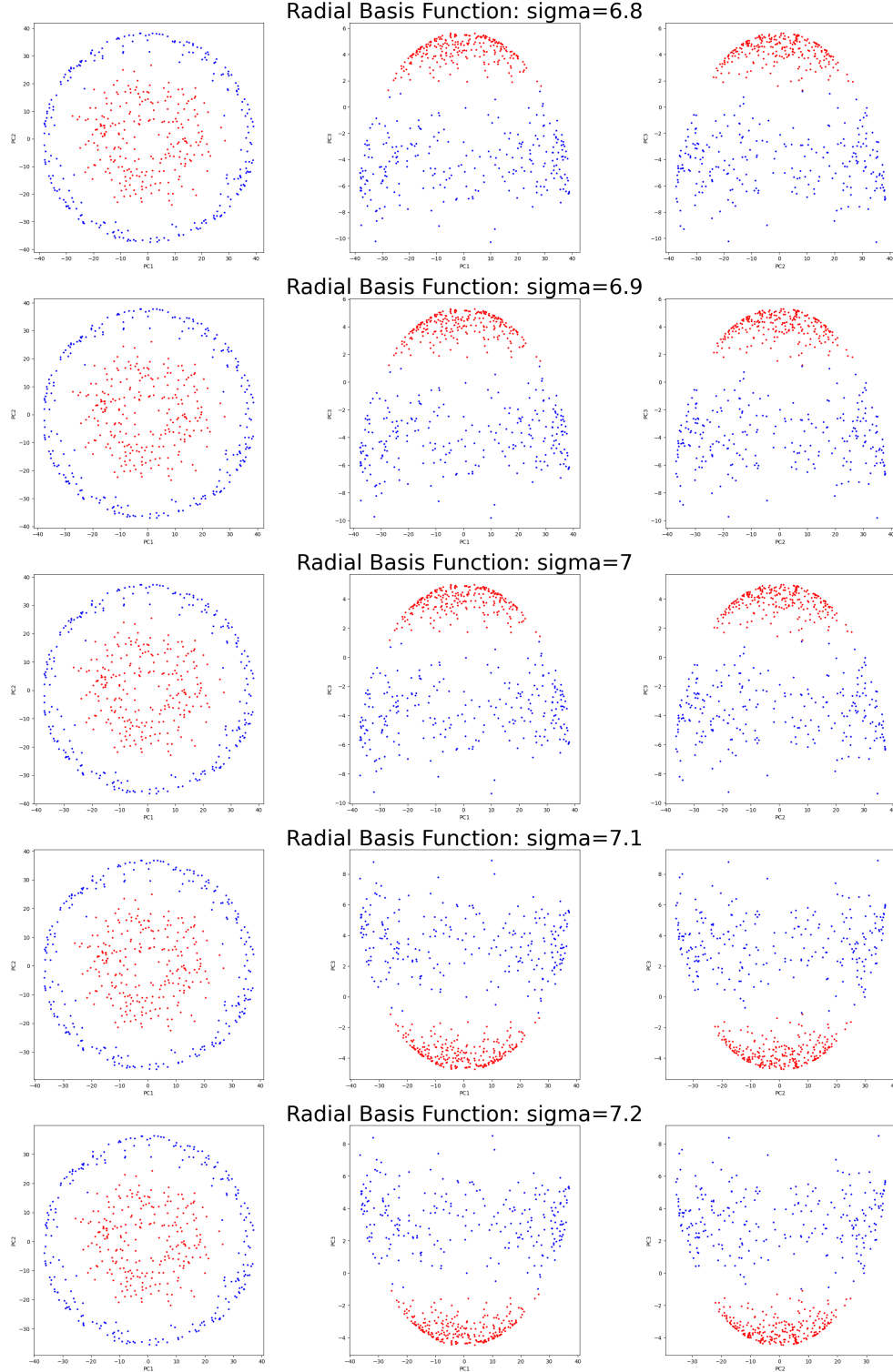


Figure 8: 2D plots of radial basis function kernels of different hyper parameters around the linear separation threshold.